Unit 1: Introduction

By
Dr.Sreeparna Chakrabarti
Faculty Member
Department of Computer Science
Kristu Jayanti Deemed to be University
Bangalore

Unit 1: Introduction **07 hrs**

Overview of C - Importance of C; Basic structure of a C program. Constants, Variables and Data types - C Tokens; Keywords and identifiers; Constants; Variables; Data types; Declaration and initialization of variables; Symbolic constants. Input and Output: Formatted Input Output function-scanf(), printf(); Reading and writing characters

COMPUTER

Definition: A machine that processes input to produce output.

Main Units:

Input (Keyboard, Mouse)

CPU

Memory (RAM-primary memory & volatile, ROM-non-volatile, secondary-non-volatile &user data is stored)

Output (Monitor, Printer)

WHAT IS SOFTWARE VS. HARDWARE?

Hardware: Physical parts of a computer

Software: Set of programs that instruct the hardware

Examples:

Hardware: Keyboard, Mouse, CPU

Software: MS Word, Browsers, Operating Systems

WHAT IS PROGRAMMING?

Definition: Giving instructions to a computer to perform tasks.

Why programming?

Automate tasks

Solve real-world problems

Build websites, games, software, etc.

UNDERSTANDING ALGORITHMS AND FLOWCHARTS

Algorithm: Step-by-step instructions to solve a problem

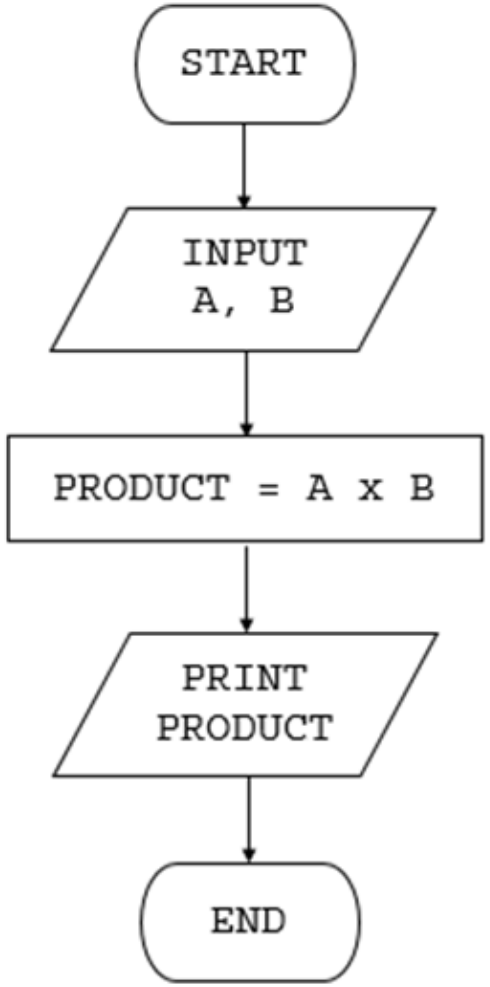
Example: Algorithm to make tea or calculate the sum of two numbers

Flowchart: A visual representation of an algorithm

Symbols: Start/End (Oval), Process (Rectangle), Decision (Diamond)

UNDERSTANDING ALGORITHMS AND FLOWCHARTS

INPUT AND PRINT THE PRODUCT OF 2 NUMBERS



UNDERSTANDING ALGORITHMS AND FLOWCHARTS

```
Step 1: Start
Step 2: Declare variables 'num1', 'num2', and
'sum'.
Step 3: Read the value of 'num1'.
Step 4: Read the value of 'num2'.
Step 5: Calculate the sum: 'sum' = 'num1' +
'num2'.
Step 6: Display the value of 'sum'.
Step 7: Stop
```

TYPES OF PROGRAMMING LANGUAGES

- **Low-Level Languages**: Machine language,Assembly (hard to read/write)
- **High-Level Languages**: C(considered middle level), Python, Java (easier to understand)

🧠 *Analogy*: Low-level is like talking to a machine in 1s and 0s; high-level is like talking in English.

WHAT IS A COMPILER AND INTERPRETER?

Compiler: Converts entire code into machine language at once (e.g., C)

Interpreter: Converts one line at a time (e.g., Python)

🧠 In C, you write code → compile → run → get output.

STEPS IN WRITING A PROGRAM

Understand the problem
Write the algorithm
Convert it into code (C language)
Compile it
Debug errors
Run the program
Get output

SHOULD I KNOW ANYTHING ELSE?

What is an Operating System (OS)?

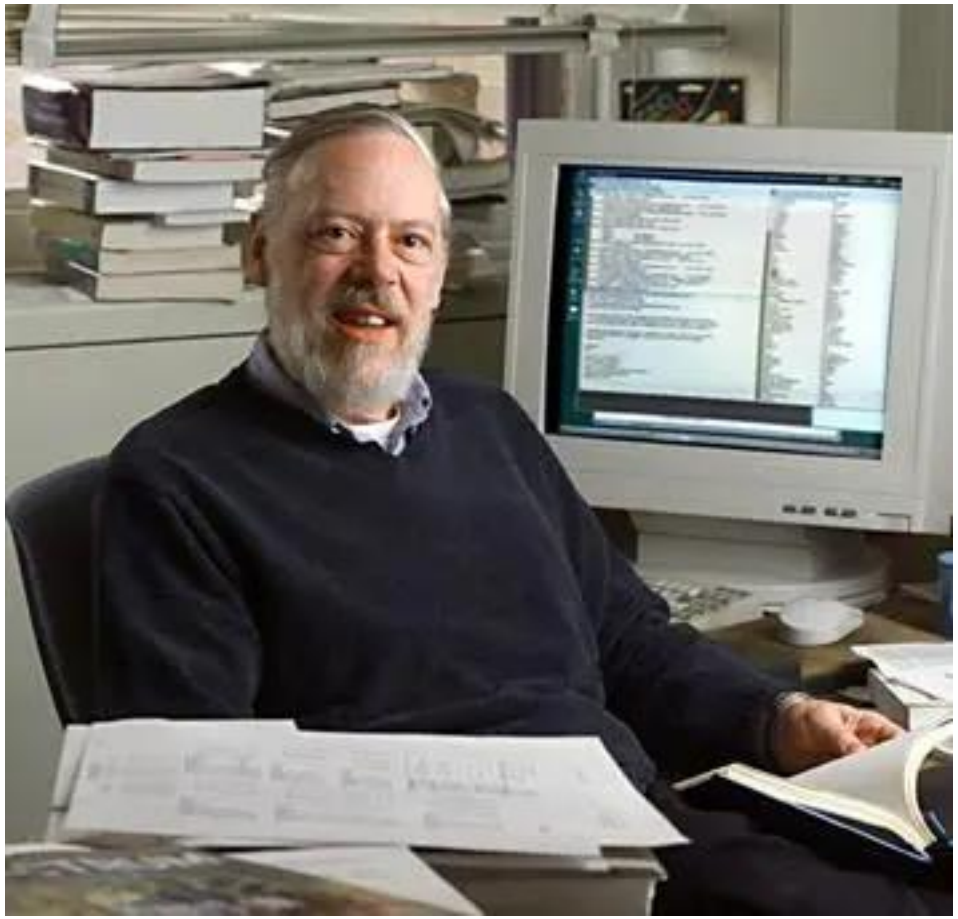
Definition: Software that manages hardware and software

Examples: Windows, Linux, MacOS

Why it matters? C programs are run on top of the OS.

INTRODUCTION

The C Language is developed by Dennis Ritchie.



HISTORY OF C LANGUAGE

C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A.

Dennis Ritchie is known as the **founder of the c language**.

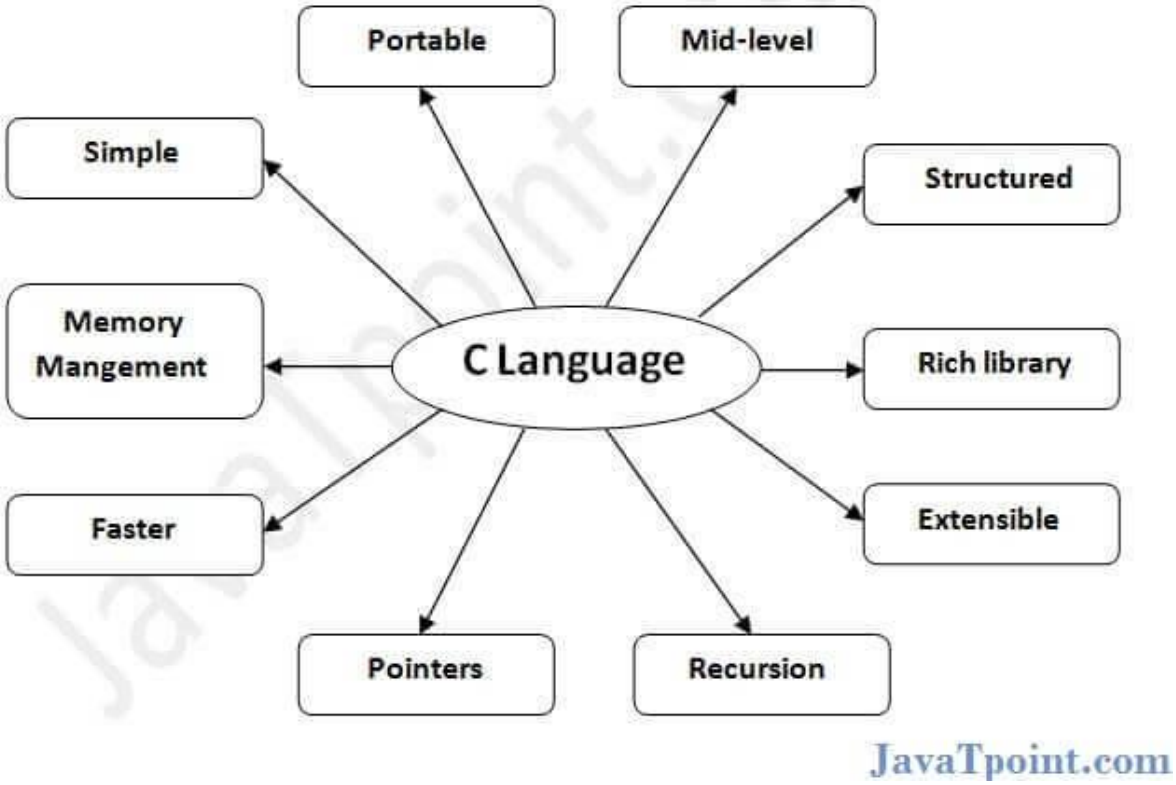
It was developed to overcome the problems of previous languages such as B, BCPL, etc.

IMPORTANCE OF C LANGUAGE

C is the widely used language. It provides many **features** that are given below.

Simple

- Machine Independent or Portable
- Mid-level programming language
- structured programming language
- Rich Library
- Memory Management
- Fast Speed
- Pointers
- Recursion
- Extensible



IMPORTANCE OF C LANGUAGE

1) Simple

C is a simple language in the sense that it provides a **structured approach** (to break the problem into parts), **the rich set of library functions, data types,** etc.

2) Machine Independent or Portable

Unlike assembly language, C programs **can be executed on different machines** with some machine specific changes. Therefore, C is a machine independent language.

3) Mid-level programming language

C has low level language capabilities (close interaction with hardwares) and high level features (structured programming). That is why it is known as mid-level language.

IMPORTANCE OF C LANGUAGE

4) Structured programming language

C is a structured programming language in the sense that **we can break the program into parts using functions**. So, it is easy to understand and modify. Functions also provide code reusability.

5) Rich Library

C **provides a lot of inbuilt functions** that make the development fast.

6) Memory Management

It supports the feature of **dynamic memory allocation**. In C language, we can free the allocated memory at any time by calling the **free()** function.

IMPORTANCE OF C LANGUAGE

7) Speed

The compilation and execution time of C language is fast since there are lesser inbuilt functions and hence the lesser overhead.

8) Pointer

C provides the feature of pointers. We can directly interact with the memory by using the pointers. We **can use pointers for memory, structures, functions, array**, etc.

9) Recursion

In C, we **can call the function within the function**. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

IMPORTANCE OF C LANGUAGE

10) Extensible

C language is extensible because it **can easily adopt new features.**

FIRST C PROGRAM

To write the first c program, open the C console and write the following code:

```
#include <stdio.h>
//now the main function starts
void main(){
printf("Hello C Language");
}
```

#include <stdio.h> includes the **standard input output** library functions. The printf() function is defined in stdio.h .

void main() :The **main() function is the entry point of every program** in c language.

printf() The printf() function is **used to print data** on the console.

BASIC STRUCTURE OF A C PROGRAM

Documentation section

Link section

Definition section

Global declaration section

main () Function section

{

Declaration part
Executable part

}

Subprogram section

Function 1
Function 2
……..
……..
Function n

(User defined functions)

Basic Structure of C Program

BASIC STRUCTURE OF A C PROGRAM

Documentation section

The documentation section consists of a set of comment lines giving the name of the program, the author and other details, which the programmer would like to use later.

Link section

The link section provides instructions to the compiler to link functions from the system library such as using the #include directive.

Definition section

The definition section defines all symbolic constantsusing the #define directive.

Global declaration section

There are some variables that are used in more than one function. Such variables are called global variables and are declared in the global declaration section that is outside of all the functions. This section also declares all the user-defined functions.

BASIC STRUCTURE OF A C PROGRAM

main () function section

Every C program must have one main function section. This section contains two parts; declaration part and executable part- these two parts must appear between the opening and closing braces.

Declaration part

The declaration part declares all the variables used in the executable part.

Executable part

There is at least one statement in the executable part.

The program execution begins after the main() at the opening brace and ends at the closing brace. The closing brace of the main function is the logical end of the program.

Most statements in the declaration and executable part end with a semicolon.

BASIC STRUCTURE OF A C PROGRAM

Subprogram section

If the program is a multi-function program then the subprogram section contains all the user-defined functions that are called in the main () function.

User-defined functions are generally placed immediately after the main () function, although they may appear in any order.

All section, except the main () function section may be absent when they are not required.

BASIC STRUCTURE OF A C PROGRAM

<u>main() function</u>

- The main function is an integral part of the programming languages such as C, C++, and Java.
- The main function in C is the entry point of a program where the execution of a program starts.
- It is a user-defined function that is mandatory for the execution of a program because when a C program is executed, the operating system starts executing the statements in the main() function.
- Every program has exactly one main function.
- The name of this function should be “main” not anything else.
- The main function always returns an integer value or void.
- The main function is called by OS, not the user.

PROGRAM TO PRINT CUBE OF GIVEN NUMBER

```
#include<stdio.h>
int main(){
int number;
printf("enter a number:");
scanf("%d",&number);
printf("cube of number is:%d ",number*number*number);
return 0;
}
```

Output

```
enter a number:5
cube of number is:125
```

PROGRAM TO PRINT CUBE OF GIVEN NUMBER

The **scanf("%d",&number)** statement reads integer number from the console and stores the given value in number variable.

The **printf("cube of number is:%d",number*number*number)** statement prints the cube of number on the console.

PROGRAM TO PRINT SUM OF 2 NUMBERS

```
#include<stdio.h>
int main(){
int x=0,y=0,result=0;

printf("enter first number:");
scanf("%d",&x);
printf("enter second number:");
scanf("%d",&y);

result=x+y;
printf("sum of 2 numbers:%d ",result);

return 0;
}
```

Output

```
enter first number:9
enter second number:9
sum of 2 numbers:18
```

Tokens

- A token in C can be defined as the smallest individual element of the C programming language that is meaningful to the compiler.
- It is the basic component of a C program.

Tokens

The tokens of C language can be classified into six types based on the functions they are used to perform. The types of C tokens are as follows:

1. **Keywords**
2. **Identifiers**
3. **Constants**
4. **Strings**
5. **Special Symbols**
6. **Operators**

Tokens - Keywords

- The keywords are pre-defined or reserved words in a programming language.
- Each keyword is meant to perform a specific function in a program.
- Since keywords are referred names for a compiler, they can’t be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed.
- You cannot redefine keywords

Tokens - Keywords

- **C** language supports **32** keywords which are given below:

auto	**double**	**int**	**struct**
break	**else**	**long**	**switch**
case	**enum**	**register**	**typedef**
char	**extern**	**return**	**union**
const	**float**	**short**	**unsigned**
continue	**for**	**signed**	**void**
default	**goto**	**sizeof**	**volatile**
do	**if**	**static**	**while**

Tokens - Identifiers

- Identifiers are used as the general terminology for the naming of variables, functions, and arrays.
- These are user-defined names consisting of an arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character.
- Identifier names must differ in spelling and case from any keywords. You cannot use keywords as identifiers; they are reserved for special use.
- Once declared, you can use the identifier in later program statements to refer to the associated value.
- A special identifier called a statement label can be used in goto statements.

Tokens - Constants

- The constants refer to the variables with fixed values.
- They are like normal variables but with the difference that their values can not be modified in the program once they are defined.
- Constants may belong to any of the data types

Tokens - Strings

- Strings are nothing but an array of characters ended with a null character (‘\0’).
- This null character indicates the end of the string. Strings are always enclosed in double quotes.
- Whereas, a character is enclosed in single quotes in C and C++.

Tokens – Special Characters

The following special symbols are used in C having some special meaning and thus, cannot be used for some other purpose.

Some of these are listed below:

- **Brackets[]:** Opening and closing brackets are used as array element references. These indicate single and multidimensional subscripts.
- **Parentheses():** These special symbols are used to indicate function calls and function parameters.
- **Braces{}:** These opening and ending curly braces mark the start and end of a block of code containing more than one executable statement.

Tokens – Special Characters

- **Comma (,):** It is used to separate more than one statement like for separating parameters in function calls.
- **Colon(:):** It is an operator that essentially invokes something called an initialization list.
- **Semicolon(;):** It is known as a statement terminator. It indicates the end of one logical entity. That's why each individual statement must be ended with a semicolon.
- **Asterisk (*):** It is used to create a pointer variable and for the multiplication of variables.

Tokens – Special Characters

- **Assignment operator(=):** It is used to assign values and for logical operation validation.
- **Pre-processor (#):** The preprocessor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.
- **Period (.):** Used to access members of a structure or union.
- **Tilde(~):** Bitwise One's Complement Operator.

Tokens - Operators

Operators are symbols that trigger an action when applied to C variables and other objects.

The data items on which operators act are called operands.

Depending on the number of operands that an operator can act upon, operators can be classified as follows:

- **Unary Operators:** Those operators that require only a single operand to act upon are known as unary operators.
- For Example increment and decrement operators.

Tokens - Operators

- **Binary Operators:** Those operators that require two operands to act upon are called binary operators.Binary operators can further are classified into:
  - Arithmetic operators
  - Relational Operators
  - Logical Operators
  - Assignment Operators
  - Bitwise Operator

- **Ternary Operator**: The operator that requires three operands to act upon is called the ternary operator. Conditional Operator(?:) is also called the ternary operator.

VARIABLES IN C

A **variable** is a name of the memory location.

It is used to store data. Its value can be changed, and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

type variable_list;

```
int a;
float b;
char c;
```

Here, a, b, c are variables. The int, float, char are the data types.

We can also provide values while declaring the variables as given below:

```
int a=10,b=20;//declaring 2 variable of integer type
float f=20.8;
char c='A';
```

RULES FOR DEFINING VARIABLES

- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Valid variable names:

```
int a;
int _ab;
int a30;
```

Invalid variable names:

```
int 2;
int a b;
int long;
```

TYPES OF VARIABLES IN C

There are many types of variables in c:

- local variable
- global variable
- static variable
- automatic variable
- external variable

LOCAL VARIABLE

A variable that is declared inside the function or block is called a local variable.

It must be declared at the start of the block.

```
void function1(){
int x=10;//local variable
}
```

You must have to initialize the local variable before it is used.

GLOBAL VARIABLE

A variable that is declared outside the function or block is called a global variable.

Any function can change the value of the global variable. It is available to all the functions.

It must be declared at the start of the block.

```
int value=20;//global variable
void function1(){
int x=10;//local variable
}
```

STATIC VARIABLE

A variable that is declared with the static keyword is called static variable.

It retains its value between multiple function calls.

```
void function1(){
int x=10;//local variable
static int y=10;//static variable
x=x+1;
y=y+1;
printf("%d,%d",x,y);
}
```

STATIC VARIABLE

```
#include<stdio.h>
void function1() {
        int x=10;//local variable
        static int y=10;//static variable
        x=x+1;
        y=y+1;
        printf("%d,%d\n",x,y);
}
void main() {
        function1();
        function1();
        function1();
}
```

OUTPUT

11,11
11,12
11,13

AUTOMATIC VARIABLE

All variables in C that are declared inside the block, are automatic variables by default.

We can explicitly declare an automatic variable using **auto keyword**.

```
void main(){
int x=10;//local variable (also automatic)
auto int y=20;//automatic variable
}
```

EXTERNAL VARIABLE

We can share a variable in multiple C source files by using an external variable. To declare an external variable, you need to use **extern keyword.**

So extern is mainly used for **global variables** that are shared across **multiple source files.**

```
#include <stdio.h>
extern int a;
void main()
{
int a=2;
printf("%d", a);
}
```

EXTERNAL VARIABLE

📁 **file1.c**

```c
#include <stdio.h>
int a = 10;    // definition
```

📁 **file2.c**

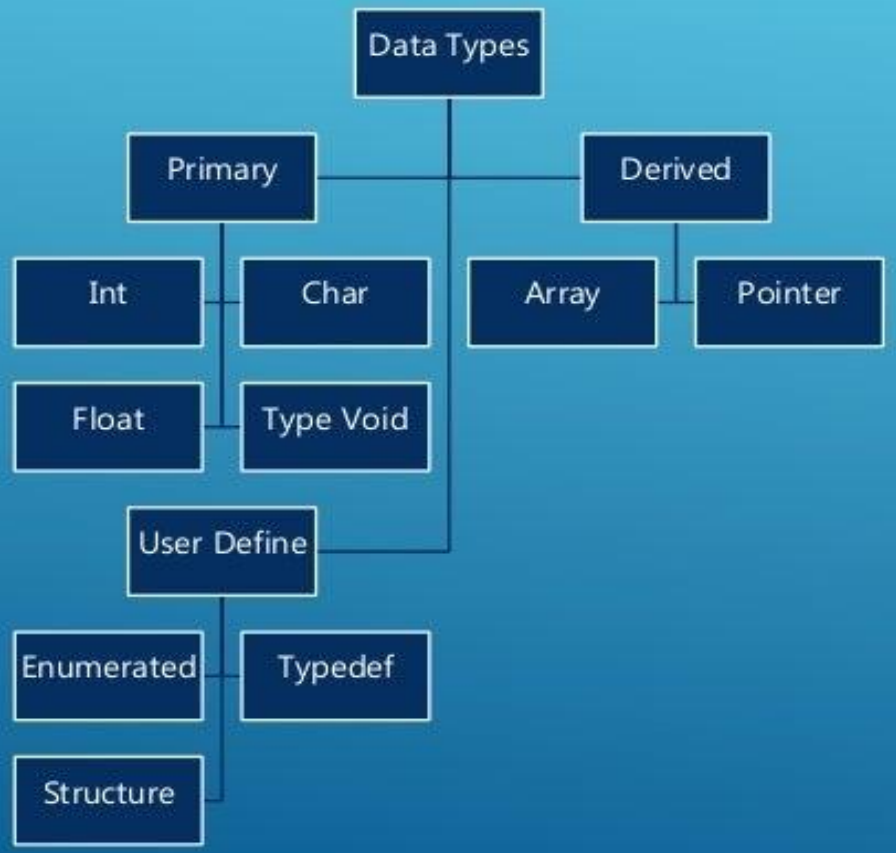
```c
#include <stdio.h>
extern int a;    // declaration
int main() {
    printf("%d", a);   // uses 'a' from file1.c
    return 0;
}
```

DATA TYPES

DATA TYPES

- In the C Programming language, data types refer to a broad system used for declaring variables or functions of different types. The type of a variable determines how much space it occupies in storage and how the bit pattern stored is interpreted.

TYPES OF DATA TYPES



INT DATA TYPE

- Integers are whole numbers with a range of values, range of values are machine dependent. Generally an integer occupies 2 bytes memory space and its value range limited to -32768 to +32767.

Data Type	Control String	Size	Range
int	%d	2 bytes	-32768 to +32767
long int	%ld	4 bytes	-2^{31} to $+2^{31}$
unsigned int	%u	2 bytes	0 to 65535

CHAR DATA TYPE

- Character type variable can hold a single character. As there are singed and unsigned int (either short or long), in the same way there are signed and unsigned chars; both occupy 1 byte each, but having different ranges. Unsigned characters have values between 0 and 255; signed characters have values from –128 to 127.

CHARACTER DATA TYPE

(1)

```
#include <stdio.h>
void main()
{
    char i = 'd';
    printf(”char: %c\n", i);
}
```

(2)

```
#include <stdio.h>
void main()
{

    int chr = 97;
    unsigned char i = chr;
    printf("unsigned char: %c\n", i);
}
```

ASCII

```
cook@pop-os:~$ ascii -d
    0 NUL    16 DLE    32       48 0    64 @    80 P    96 `   112 p
    1 SOH    17 DC1    33 !     49 1    65 A    81 Q    97 a   113 q
    2 STX    18 DC2    34 "     50 2    66 B    82 R    98 b   114 r
    3 ETX    19 DC3    35 #     51 3    67 C    83 S    99 c   115 s
    4 EOT    20 DC4    36 $     52 4    68 D    84 T   100 d   116 t
    5 ENQ    21 NAK    37 %     53 5    69 E    85 U   101 e   117 u
    6 ACK    22 SYN    38 &     54 6    70 F    86 V   102 f   118 v
    7 BEL    23 ETB    39 '     55 7    71 G    87 W   103 g   119 w
    8 BS     24 CAN    40 (     56 8    72 H    88 X   104 h   120 x
    9 HT     25 EM     41 )     57 9    73 I    89 Y   105 i   121 y
   10 LF     26 SUB    42 *     58 :    74 J    90 Z   106 j   122 z
   11 VT     27 ESC    43 +     59 ;    75 K    91 [   107 k   123 {
   12 FF     28 FS     44 ,     60 <    76 L    92 \   108 l   124 |
   13 CR     29 GS     45 -     61 =    77 M    93 ]   109 m   125 }
   14 SO     30 RS     46 .     62 >    78 N    94 ^   110 n   126 ~
   15 SI     31 US     47 /     63 ?    79 O    95 _   111 o   127 DEL
```

FLOAT DATA TYPE

- The float data type is used to store fractional numbers (real numbers) with 6 digits of precision. Floating point numbers are denoted by the keyword float. When the accuracy of the floating point number is insufficient, we can use the double to define the number. The double is same as float but with longer precision and takes double space (8 bytes) than float. To extend the precision further we can use long double which occupies 10 bytes of memory space.

Data Type	Size	Value Range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 Decimal Places
double	8 byte	2.3E-308 to 1.7E+308	15 Decimal Places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 Decimal Places

FLOAT DATA TYPE

Example1:

```
#include <stdio.h>
void main()
{
    float f = 3.50;
    printf(”f: %f\n", f);
}
```

Example2:

```
#include <stdio.h>
void main() {
// Example with double
double d = 3.50;
printf("d: %lf\n", d); // Use %lf for double

// Example with long double
long double ld = 3.50L; // Use 'L' suffix for long double literal
printf("ld: %Lf\n", ld); // Use %Lf for long double
}
```

COMMON FORMAT SPECIFIERS

Format	Type
%d	Integer
%f	Float
%c	Character
%s	String
%lf	Double
%Lf	Long Double

TYPE VOID DATA TYPE

- The void type has no values therefore we cannot declare it as variable as we did in case of integer and float. The void data type is usually used with function to specify its type. Like in our first C program we declared "main ()" as void type because it does not return any value.

DERIVED DATA TYPE

- **Array:** An array in C language is a collection of similar data-type, means an array can hold value of a particular data type for which it has been declared. Arrays can be created from any of the C data-types int.
- **Pointer: C** Pointer is a special variable that can be used to store address of another variable.

ENUMERATED DATA TYPE (ENUM)

Enumerated data type is a user defined data type having finite set of enumeration constants. The keyword 'enum' is used to create enumerated data type. Enumeration data type consists of named integer constants as a list. It start with 0 (zero) by default and value is incremented by 1 for the sequential identifiers in the list.

Syntax:

Enum [data_type] {const1, const2... constn};

Enum example in C:

enum month { Jan, Feb, Mar }; or /* Jan, Feb and Mar variables will be assigned to 0, 1 and 2 respectively by default */

enum month { Jan = 1, Feb, Mar }; /* Feb and Mar variables will be assigned to 2 and 3 respectively by default */

enum month { Jan = 20, Feb, Mar }; /* Jan is assigned to 20. Feb and Mar variables will be assigned to 21 and 22 respectively by default */

EMUNERATED DATA TYPE

(1)

```
#include <stdio.h>
 void main()
{
  enum weekdays{Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
  printf("The value of 1st day is %d \n",Sunday);
  printf("The value of 3rd day is %d",Tuesday);
}
```

The value of 1st day is 0

The value of 3rd day is 2

(2)

```
#include <stdio.h>
 void main()
{
  enum weekdays{Sunday=3, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
  printf("The value of 1st day is %d \n",Sunday);
  printf("The value of 3rd day is %d",Tuesday);
}
```

The value of 1st day is 3

The value of 3rd day is 5

TYPEDEF DATA TYPE

It is used to create new data type. But it is commonly used to change existing data type with another name.

Syntax:

typedef [data_type] new_data_type;

Example:

typedef int integer;

integer roll_no;

STRUCTURE DATA TYPE

C Structure is a collection of different data types which are grouped together and each element in a C structure is called member.

Example:

```
struct student {
    int roll_no;
    char name[20];
    char city[20];
}
```

STRUCTURE DATA TYPE

```
#include <stdio.h>
struct Product {
  int productID; // Product ID (integer)
  float price;   // Price (floating-point number)
  char grade;    // Product grade (single character)
};
void main() {
  struct Product item1;
  item1.productID = 12345;
  item1.price = 25.99;
  item1.grade = 'A’;
  printf("Product Details:\n");
  printf("Product ID: %d\n", item1.productID);
  printf("Price: %.2f\n", item1.price);
  printf("Grade: %c\n", item1.grade);
}
```

C - CONSTANTS AND LITERALS

CONSTANTS

- Constants refer to fixed values that the program may not alter during its execution.
- Constants can be of any of the basic data types like *an integer constant, a floating constant, a character constant, or a string literal*.
- There are enumeration constants as well.
- Constants are treated just like regular variables except that their values cannot be modified after their definition.
- <u>Example:</u>

```
#include <stdio.h>
void main()
{
    const int int_const = 25;
    printf("Printing value of Integer Constant: %d\n",int_const);
    //int_const=30; is not allowed
}
```

LITERALS

- In C, Literals are the constant values that are assigned to the variables.
- Literals represent fixed values that cannot be modified.
- Literals contain memory but they do not have references as variables.
- Unlike variables, literals are not names that point to memory locations.
- Generally, both terms, constants, and literals are used interchangeably.
- For example, “const int a= 5;“, is a constant expression and the value 5 is referred to as a constant integer literal.
- In “int y=5”, 5 is a literal and also considered a constant in this context as its value does not change during the execution of the program.

CHARACTER CONSTANTS

Character literals are enclosed in single quotes, e.g., 'x' can be stored in a simple variable of **char** type.

A character literal can be a plain character (e.g., 'x'), an escape sequence (e.g., '\t').

Constants	Meaning
\a	beep sound
\b	backspace
\f	form feed
\n	new line
\r	carriage return
\t	horizontal tab
\v	vertical tab
\'	single quote
\"	double quote
\\	backslash
\0	null

```
#include <stdio.h>
void main() {
   printf("Hello\tWorld\n\n");
}
```

When the above code is compiled and executed, it produces the following result –

```
Hello    World
```

STRING LITERALS

String literals or constants are enclosed in double quotes "".

A string contains characters that are similar to character literals: plain characters, escape sequences, and universal characters.

You can break a long line into multiple lines using string literals and separating them using white spaces.

Here is an example of string literals:

"hello, dear"

DEFINING CONSTANTS

There are two simple ways in C to define constants –

- Using **#define** preprocessor-symbolic constant
- Using **const** keyword.

THE #DEFINE PREPROCESSOR

Given below is the form to use #define preprocessor to define a constant –

#define identifier value

```
#include <stdio.h>

#define LENGTH 10
#define WIDTH  5
#define NEWLINE '\n'

int main() {
   int area;

   area = LENGTH * WIDTH;
   printf("value of area : %d", area);
   printf("%c", NEWLINE);

   return 0;
}
```

When the above code is compiled and executed, it produces the following result –

```
value of area : 50
```

THE CONST KEYWORD

You can use const prefix to declare constants with a specific type as follows –

const type variable = value;

```
#include <stdio.h>

int main() {
   const int  LENGTH = 10;
   const int  WIDTH = 5;
   const char NEWLINE = '\n';
   int area;

   area = LENGTH * WIDTH;
   printf("value of area : %d", area);
   printf("%c", NEWLINE);

   return 0;
}
```

When the above code is compiled and executed, it produces the following result –

```
value of area : 50
```

Formatted and Unformatted Input/Output functions in C

- Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user.
- These types of I/O functions can help to display the output to the user in different formats using the format specifiers.

1	%d	int/signed int	used for I/O signed integer value
2	%c	char	Used for I/O character value
3	%f	float	Used for I/O decimal floating-point value
4	%s	string	Used for I/O string/group of characters
5	%ld	long int	Used for I/O long signed integer value
6	%u	unsigned int	Used for I/O unsigned integer value
7	%i	unsigned int	used for the I/O integer value
8	%lf	double	Used for I/O fractional or floating data
9	%n	prints	prints nothing

- The following formatted I/O functions will be discussed in this section-

- printf()
- scanf()

printf():

- printf() function is used in a C program to display any value like float, integer, character, string, etc on the console screen.
- It is a pre-defined function that is already declared in the stdio.h(header file).
- Syntax 1:
- To display any variable value.
- printf("Format Specifier", var1, var2, …., varn);

scanf():

- scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any data type like integer, float, character, string, and many more.
- This function is declared in stdio.h(header file), that's why it is also a pre-defined function.
- In scanf() function we use &(address-of operator) which is used to store the variable value on the memory location of that variable.

- Syntax:
- scanf(“Format Specifier”, &var1, &var2, …., &varn);

Example

Reading and writing characters: using unformatted I/O

- Unformatted I/O functions are used only for character data type or character array/string and cannot be used for any other datatype.

- These functions are used to read single input from the user at the console and it allows to display the value at the console.

Why they are called unformatted I/O?

- These functions are called unformatted I/O functions because we cannot use format specifiers in these functions and hence, cannot format these functions according to our needs.
- getch()
- getche()
- getchar()
- putchar()
- putch()

getch()

- function reads a single character from the keyboard by the user but doesn't display that character on the console screen and immediately returned without pressing enter key.
- This function is declared in conio.h(header file).
- getch() is also used for hold the screen.

- *getch();*
- *or*
- *variable-name = getch();*

getche():

- function reads a single character from the keyboard by the user and displays it on the console screen and immediately returns without pressing the enter key.
- This function is declared in conio.h(header file).

- *getche();*
- *or*
- *variable_name = getche();*

```c
// C program to implement
// the getche() function
#include <conio.h>
#include <stdio.h>

// Driver code
int main()
{
    printf("Enter any character: ");

    // Reads a character and
    // displays immediately
    getche();
    return 0;
}
```

Output:

Enter any character: g

getchar():

- function is used to read only a first single character from the keyboard whether multiple characters is typed by the user and this function reads one character at one time until and unless the enter key is pressed.

- This function is declared in stdio.h(header file)

- *Variable-name = getchar();*

```c
// C program to implement
// the getchar() function
#include <conio.h>
#include <stdio.h>

// Driver code
int main()
{
    // Declaring a char type variable
    char ch;

    printf("Enter the character: ");

    // Taking a character from keyboard
    ch = getchar();

    // Displays the value of ch
    printf("%c", ch);
    return 0;
}
```

Output:

Enter the character: a

a

putchar()

- function is used to display a single character at a time by passing that character directly to it or by passing a variable that has already stored a character.
- This function is declared in stdio.h(header file)

- ***putchar(variable_name);***

```c
// C program to implement
// the putchar() function
#include <conio.h>
#include <stdio.h>

// Driver code
int main()
{
    char ch;
    printf("Enter any character: ");

    // Reads a character
    ch = getchar();

    // Displays that character
    putchar(ch);
    return 0;
}
```

Output:

Enter any character: Z

Z

putch():

- putch() function is used to display a single character which is given by the user and that character prints at the current cursor location.
- This function is declared in conio.h(header file)
- Syntax:
- putch(variable_name);

```c
// C program to implement
// the putch() functions
#include <conio.h>
#include <stdio.h>

// Driver code
int main()
{
    char ch;
    printf("Enter any character:\n ");

    // Reads a character from the keyboard
    ch = getch();

    printf("\nEntered character is: ");

    // Displays that character on the console
    putch(ch);
    return 0;
}
```

Thank you!